

R⁴ Holographic Graph Compiler

Implementation Plan, Runtime Contract, and Formalized Proof Obligations

Date: July 2026

Status: research implementation plan and formalized proof obligations. Structural runtime claims are distinguished from empirical teacher-fidelity claims.

Executive Summary

This document formalizes the research direction for compiling a finely trained Hugging Face model into a transformer-free, hologram-like semantic graph. The source transformer remains available only during offline compilation. The deployed runtime performs bounded activation, traversal, and next-token prediction over immutable graph artifacts using CPU-native bitwise and integer operations.

The target is not a hand-authored linguistic model and not a smaller transformer. It is an unsupervised behavioral cross-compiler: it discovers multiresolution semantic regions, overlapping memberships, transition edges, and token-emission residuals from the trained model's representations and outputs. Linguistic frameworks may be used as evaluation probes or post-hoc annotations, but they are not required supervision. The compiler must also defend against semantic degeneracy: predictive clusters that merely memorize phrases, tokenizer artifacts, or narrow corpus correlations must not be accepted as reusable semantic regions without invariance and sufficiency evidence.

The runtime contract is deliberately narrow: no transformer layers, no matrix multiplication, no floating-point requirement, no heap allocation in the forward-pass hot path, no unbounded graph search, and no mutable global state. Each inference step operates over caller-owned fixed-capacity buffers and packed memory-mappable tables using XOR, AND, OR, shifts, rotates, popcount, integer addition, comparisons, and contiguous table reads.

The proof section establishes what can be proven from the design itself—allocation freedom, operation-set conformance, determinism, bounded work, artifact integrity, and witness replay—and separates those properties from empirical fidelity to the teacher, which must be certified by held-out measurements rather than asserted as mathematical identity.

1. Scope and Research Thesis

Primary thesis: a trained neural model is one encoding of a conditional behavior. R⁴ compilation derives a different encoding of that behavior as an immutable, overlapping, multiresolution graph. The graph is optimized for deterministic CPU inference and proof-carrying traversal.

The semantic graph is hologram-like in the operational sense: meaning is distributed across multiple measurements; a short route reconstructs a coarse semantic cloud; additional route decisions increase fidelity; partial graph activation remains useful; and the same graph supports refinement, abstraction, lateral expansion, forward transition, and reverse evidence lookup.

The compiler is unsupervised with respect to external labels. It may inspect source embeddings, contextual hidden states, logits, and controlled perturbation responses. It may run clustering, quantization, Boolean synthesis, and expensive numerical optimization. It must not require manually labeled parts of speech, parse trees, frames, ontologies, or semantic classes to produce a deployable artifact.

2. Non-Goals

R⁴ does not claim exact equivalence to the teacher on all possible inputs. The compilation target and teacher belong to different function classes, so fidelity is a measured, bounded property over declared evaluation distributions.

The project does not assign indivisible semantic atoms to words, tokens, or objects. A token occurrence can activate multiple regions across multiple resolutions and can resolve differently under different contexts.

The runtime will not execute a general-purpose parser, arbitrary graph database query engine, dynamic allocator, hash-map-based search, or Monte Carlo grammar sampler. Such mechanisms may appear in offline research tooling but not in the deployed prediction kernel.

Cryptographic content identifiers are not semantic hashes. Content CIDs preserve identity and provenance; semantic route codes are separately compiled, versioned, and intentionally locality-preserving.

3. System Boundary

The system has three strictly separated phases:

Compiler phase: offline, potentially expensive, and allowed to use floating point, matrix multiplication, GPU/CPU accelerators, clustering, optimization, and model introspection. Every output is canonicalized and content-addressed.

Runtime phase: per-token inference over frozen graph artifacts. Only the declared operation kernel is available. All state is supplied by the caller or resides in fixed-capacity stack or preallocated buffers.

Certifier phase: evaluates teacher fidelity, semantic-neighbor preservation, graph stability, operation counts, allocation behavior, and artifact integrity. It may use expensive instrumentation but never participates in inference.

4. Conceptual Model

A contextual observation—not an isolated token—is the primary compilation sample. The same token in different contexts may occupy different regions. Each observation is assigned a bounded set of memberships at each resolution.

A semantic region is a predictively coherent area of the teacher's behavior. It is represented at runtime by a compact prototype, comparison mask, calibrated acceptance radius, and packed edge ranges. Human-readable labels are optional metadata and do not affect inference.

The semantic graph is a directed acyclic refinement graph with cross-links. Parent and child edges implement zoom-out and zoom-in. Overlap and neighbor edges implement lateral expansion. Forward and reverse transition indexes describe likely semantic successors and teacher-supported predecessors. Multiple parents are allowed, so the structure is not restricted to a tree.

Next-token prediction is factored into semantic transition and lexical emission: the active cloud predicts a bounded next semantic cloud; region-to-token residual tables then score likely lexical realizations. Exact-context residual evidence handles behavior not explained by the compressed graph.

5. Mathematical Definitions

Let V be the source vocabulary, C the set of bounded token contexts, and $\Delta(V)$ the probability simplex over V . The teacher is a deterministic evaluation procedure $T: C \rightarrow \Delta(V)$ for a pinned model revision and deterministic execution mode.

A compiler observation o consists of a context c , one or more teacher-derived representation vectors $h(c)$, the teacher distribution $T(c)$, and optional perturbation traces. The observation corpus O is content-addressed and split into construction and held-out certification partitions.

A compiled semantic graph $G = (N, E_r, E_o, E_f, E_b, P, M, Q)$ contains region nodes N ; refinement edges E_r ; overlap or neighbor edges E_o ; forward transition edges E_f ; reverse indexes E_b ; packed prototypes P ; masks M ; and fixed-point score tables Q .

At depth d , an observation may activate several regions. Membership is defined by a compiler-calibrated predicate such as masked Hamming distance: $\text{active}(n, x)$ iff $\text{popcount}((H(x) \text{ XOR } p_n) \text{ AND } m_n) \leq r_n$. $H(x)$ is a compiled Boolean semantic code, not a cryptographic digest.

A runtime state contains a fixed-capacity active frontier, rolling context code, bounded token shortlist, and optional witness buffer. Runtime step R_G consumes the current state and newest token and returns a new state and prediction without allocation.

6. Compiler Pipeline

Stage 0 - Source pinning and reproducibility. Require an immutable Hugging Face revision, hash every source file, record tokenizer identity, compiler version, target architecture profile, deterministic seeds, and corpus roots.

Stage 1 - Observation generation. Collect contextual states from natural corpora, teacher-generated sequences, held-out corpora, and automatically generated perturbations. Perturbations are unsupervised transformations such as token masking, span substitution, context truncation, word-order changes, modality-specific transforms, and local counterfactuals.

Stage 2 - Representation extraction. Obtain token embeddings, final hidden states, optional intermediate traces, top-k teacher logits, and next-state trajectories. Normalize and project compiler-side representations into one or more candidate semantic lanes.

Stage 3 - Multiresolution cover induction. Use recursive k-means, spherical k-means, medoid clustering, or density-aware alternatives to propose regions. Calibrate overlapping radii or top-M assignments. Split a region only when the split improves held-out predictive coherence enough to justify its bytes and runtime work.

Stage 4 - Graph construction. Create parent/child refinement edges, explicit high-value overlap nodes, lateral neighbor edges, forward semantic transitions, and reverse transition indexes. Deduplicate equivalent subgraphs by canonical content.

Stage 5 - Boolean routing synthesis. Learn broad-to-fine Boolean observations and sparse GF(2) polynomial programs that reproduce compiler-side memberships. Synthesize decision DAGs that shortlist region candidates before exact masked-Hamming verification.

Stage 6 - Residual scoring. Convert teacher probabilities to quantized log-score residuals. The root stores base priors; child nodes store corrections relative to their parents; overlap nodes store interaction residuals only; exact-context stores capture remaining local evidence. This prevents double-counting across overlapping memberships.

Stage 7 - Packing and layout. Reorder nodes and edges for locality, flatten all graph structures into aligned arrays, generate offsets and lengths, select integer widths, and emit architecture-neutral plus optional architecture-specialized sections.

Stage 8 - Certification. Measure fidelity, semantic-neighbor preservation, stability under resampling, operation census, cache behavior, zero-allocation behavior, and witness replay. Emit a signed or CID-addressed certificate tied to the exact graph artifact.

7. Semantic Validity and Anti-Degeneracy

Unsupervised predictive clustering can converge on compressed phrase tables rather than reusable semantic structure. The compiler must therefore evaluate every proposed region along multiple independent views, not solely by next-token entropy. A region is retained as semantic structure only when it demonstrates predictive gain together with reuse across contexts, perturbation stability, and economical description of many observations.

The compiler objective should include a minimum-description-length term and multi-view invariance:

$$J(C) = \alpha * \text{next_token_gain} + \beta * \text{future_state_information} + \gamma * \text{paraphrase_consistency} + \delta * \text{perturbation_stability} + \eta * \text{cross_context_reuse} - \lambda * \text{runtime_cost} - \mu * \text{description_length}.$$

Required anti-degeneracy evaluations include paraphrase agreement, lexical substitution under preserved meaning, identical surface forms in unrelated contexts, equivalent meaning across different surface forms, context truncation, irrelevant-material reordering, cross-document recurrence, and cross-modal agreement when such data is available.

The graph must also be evaluated for predictive sufficiency. Let $R_d(c)$ be the graph state reconstructed to depth d . The compiler measures the divergence between $T(.|c)$ and $T(.|R_d(c))$ at broad, intermediate, full-cloud, and residual-augmented resolutions. This produces a rate-distortion curve relating artifact bytes and runtime operations to teacher information retained.

A semantic-coherence certificate is separate from the teacher-fidelity certificate. It reports region reuse, invariance, boundary behavior, rare-context retention, and evidence that a region generalizes beyond memorized lexical sequences.

8. Multi-Timescale Context and Persistent State

A short rolling token window cannot preserve all long-range dependencies. The runtime therefore supports a hierarchy of fixed-capacity semantic states:

token state, updated every token;

local phrase or event state, updated when local regions change;

segment state, updated at compiler-defined boundaries;

document or session state, containing a sparse durable semantic cloud;

optional external-memory references addressed by graph regions.

Each level uses compiler-generated update programs and caller-owned bounded storage. No level may grow dynamically. The compiler must learn state-compression operators that retain information useful to future teacher behavior while minimizing bytes read and operations.

Long-context certification reports fidelity by dependency distance, entity reactivation accuracy, unresolved-reference retention, topic persistence, and state saturation behavior.

9. Uncertainty, Novelty, Contradiction, and Fallback

The runtime must not interpret nearest-region assignment as evidence that an input is supported. Every step returns a resolution status derived from calibrated distances, margins, support, region disagreement, depth reached, and residual availability.

Supported indicates strong interior evidence. Boundary indicates several plausible overlapping regions. BackedOff indicates that only a broader region met support requirements. Novel indicates that no calibrated region adequately covers the input. Contradictory indicates that active regions provide materially incompatible predictions.

The manifest declares deterministic behavior for each status: continue normally, widen the candidate set, consult exact residual evidence, invoke a separately certified slower fallback, or abstain. The semantic route is never an authorization or security identity.

10. Candidate Recall and Exact Verification

Boolean routing synthesis is an optimization, not the semantic definition. The reference classifier remains the exact compiler-side region-membership procedure. A decision DAG or polynomial router may only shortlist candidates; shortlisted regions are then checked by the normative masked-Hamming verifier.

Two guarantees are tracked separately:

verifier correctness: shortlisted candidates are measured exactly under the normative integer kernel;

shortlist recall: the shortlist contains the reference-best or all reference-active regions at the declared rate.

The certificate reports top-1 and top-M region recall, false-negative rate, fallback rate, conditional fidelity without fallback, and worst observed routing error. Insufficient routing confidence triggers deterministic widening or exhaustive fallback within manifest bounds.

11. Hardware-Aware Graph Layout

The dominant runtime cost may be memory traffic rather than XOR or addition. Artifact compilation therefore includes profile-guided layout and a byte-read cost model. Nodes that co-activate should be co-located; hot edges and emission blocks should occupy contiguous cache-line-aligned ranges; cold metadata should be separated.

The optimizer evaluates structure-of-arrays versus array-of-structures layouts, local offset widths, delta encoding, shared emission blocks, branchless acceptance, prefetch strategies, memory mapping, huge-page behavior, NUMA placement, and architecture-specific vector kernels. Multiplication-free execution is not accepted as performant unless measured cache misses, branch misses, instructions, and bytes read also satisfy the performance certificate.

12. Tokenization and Model-Neutral Semantics

Source token IDs are required for teacher imitation but must not become permanent semantic identities. The compiler distinguishes source token identity, contextual span identity, semantic occurrence identity, and output lexicalization. Span- and byte-level anchors should be available so the graph does not encode accidental BPE boundaries as concepts.

The artifact format should permit a shared semantic graph with model- or tokenizer-specific emission adapters. This is optional for the first implementation but required for future cross-model, multilingual, and multimodal compilation.

13. Immutable Epochs, Patches, and Deletion

Packed base artifacts are immutable. New evidence, corrections, domain overlays, provenance deletion, and source-model upgrades are represented as content-addressed patch epochs. Each patch records its parent graph CID, new nodes and edges, score residuals, tombstones, compatibility limits, and certificate.

Runtime lookup may consult only a manifest-bounded number of layers. Periodic compaction produces a new canonical base graph. The compiler must emit route-translation evidence describing how regions from one epoch map to retained, split, merged, or removed regions in the next.

14. Safety, Bias, and Adversarial Collisions

Unsupervised compilation inherits the source model's data imbalance, unsafe associations, memorization, and tokenizer bias. Compression may amplify or erase behavior unevenly. Certification therefore measures fidelity, undesirable-behavior amplification, rare-group erasure, and provenance coverage. Evidence is retained at observation and membership-edge granularity to support traceability and deletion.

Because semantic locality intentionally creates collisions, the threat model includes crafted region activation, overlap poisoning, frontier explosion, fallback denial-of-service, integer saturation, and collision with privileged concepts. Defenses include strict fan-out and frontier limits, validated routing bytecode, checked integer semantics, bounded patch layers, adversarial collision suites, and separation of semantic routes from cryptographic identity.

15. Graph Equivalence and Statistical Claims

Two reproducibility notions are required. Byte reproducibility means identical pinned inputs and compiler configuration produce identical artifact bytes. Behavioral graph equivalence permits different internal node identities or packings when predictions, active-cloud relations, and navigation behavior agree within declared bounds on a pinned evaluation suite.

Empirical certificates include sample counts, split and corpus CIDs, confidence intervals, statistical method, distribution slices, worst-group results, seeds, and stopping criteria. Fidelity claims are limited to the declared distribution and confidence level; they are never universal-equivalence claims.

16. Runtime Architecture

The runtime maintains an active frontier rather than searching the complete graph. Each frontier entry stores a region ID, fixed-point score, membership margin, and depth. The maximum frontier width is a compile-time or artifact-declared constant.

Each token step incrementally updates a rolling Boolean or integer semantic state. The implementation must avoid reconstructing the full context representation when a shift/XOR/add recurrence can update it from the prior state, the entering token, and the expiring token.

Routing first executes a shallow decision program over the current state and active nodes. The result is a bounded shortlist of child or neighbor regions. Exact verification then performs masked XOR/popcount distance only for shortlisted regions.

Semantic transitions accumulate scores into a fixed-capacity next-frontier buffer. Emission lists from active and predicted regions accumulate token residuals into a fixed-capacity top-K structure. A root prior and exact residual store may contribute additional integer scores.

The hot path is synchronous, deterministic, allocation-free, recursion-free, and free of locks. Parallelism is reserved for compilation and batch certification unless profiling proves a benefit for a particular runtime deployment.

17. Allowed Runtime Operation Contract

The normative runtime kernel exposes only: word XOR, AND, OR, shifts, rotates, popcount, signed and unsigned integer addition/subtraction, comparisons, saturating arithmetic where declared, and indexed or sequential table reads.

No multiplication method exists in the normative kernel. Multiplication by powers of two is represented by shifts. Non-power-of-two compiler weights are converted into fixed-point residuals applied by addition, or into precomputed lookup tables when necessary.

No floating-point operation is required by the portable runtime contract. Architecture-specific accelerated implementations may use instructions only when they are proven bit-equivalent to the normative integer implementation for the artifact version.

No runtime allocation API may be reachable from prediction. Artifact parsing, memory mapping, and state construction occur before inference begins.

18. Packed Artifact Format

Use a versioned container, tentatively R4G1, with explicit endianness, integer-width, alignment, and checksum rules. Unknown mandatory sections must cause rejection; unknown optional sections may be skipped according to version policy.

Recommended sections: HEAD for model/compiler identities and limits; CODE for token codes and rolling-state programs; NODE for packed region records; EDGE for refinement, overlap, forward, and reverse edges; ROUT for decision programs, prototypes, masks, and shortlist tables; EMIT for token priors and residual lists; EXCT for exact-context residual evidence; PROV for provenance roots; CERT for certification metadata.

All offsets are relative to section starts and validated using checked arithmetic before a GraphView is created. Domain values crossing serialization boundaries use fixed-width integer types, never usize. The parser rejects out-of-range offsets, overlapping invalid ranges, unsupported feature bits, and non-canonical ordering.

The deployed view borrows immutable slices from memory-mapped or caller-owned bytes. It must not deserialize the graph into heap-resident object structures.

19. Proposed Rust Workspace

crates/uor-r4-graph-format: no_std-compatible packed types, parser, canonical serializer, checksums, versioning, and validation.

crates/uor-r4-graph-runtime: allocation-free runtime, fixed-capacity state, normative scalar kernel, architecture-specific kernels behind validated dispatch, and witness generation.

crates/uor-r4-graph-compiler: observation orchestration, clustering, overlap calibration, graph induction, residualization, Boolean synthesis, packing, and checkpoint/resume support.

crates/uor-r4-graph-certify: teacher comparison, held-out evaluation, allocation tests, operation census, cache/performance benchmarks, witness replay, and certificate emission.

crates/uor-r4-model-source: source-architecture adapters for Hugging Face models. Preserve the minimum required teacher interfaces and keep optional internal tracing separate.

crates/uor-r4-graph-cli: download, observe, compile, inspect, certify, compare, and benchmark commands. CLI concerns must not leak into core compiler or runtime crates.

crates/uor-r4-proof-model: executable specification types and property tests. Later phases may add Kani, Verus, Lean, or Coq models without coupling the production runtime to a proof tool.

20. Rust Engineering Standard

The implementation must follow the Rust development standard supplied for this project and the following operational rules.

Keep core APIs explicit and typed. Use newtypes for `NodeId`, `SectionOffset`, `TokenId`, `ScoreQ`, `Depth`, `Radius`, and artifact CIDs. Do not use sentinel values where `Option` or a dedicated enum is clearer.

Production code must not use `unwrap` or `panic` for recoverable conditions. Library boundaries return focused error enums. Application code may add context, but third-party error types must not cross into the core runtime.

Prefer borrowing over cloning. Keep fields private unless they are packed wire structures. Derive only stable, semantically appropriate traits. Avoid wildcard imports outside test modules and preludes.

Keep functions single-purpose, control flow explicit, and hot loops easy to inspect in generated assembly. Iterator abstractions are appropriate when they compile cleanly, but indexed or pointer-free slice loops are acceptable when benchmarked and clearer for the kernel.

Unsafe Rust is forbidden in the initial portable runtime. If later SIMD or memory-mapping work requires unsafe, isolate it in a small adapter module, document every safety invariant with `SAFETY` comments, add `Miri` coverage where applicable, and maintain a safe scalar reference implementation.

Workspace CI must run `cargo fmt --check`, `cargo clippy --workspace --all-targets --all-features -- -D warnings`, `cargo nextest run --workspace`, `cargo test --doc`, dependency and license audits, minimum-supported-Rust-version checks if promised, and artifact reproducibility tests.

Performance claims require criterion or `iai-callgrind`-style benchmarks, hardware metadata, stable fixtures, and regression thresholds. Never optimize solely from intuition; preserve the clear scalar reference path and compare optimized kernels against it.

21. Key Rust Types and APIs

The following types describe the intended API shape. Exact field widths must be selected from measured graph sizes and documented in the format specification.

`PackedNode` uses contiguous ranges rather than `Vec` fields. `RuntimeState` owns fixed arrays. `GraphView` borrows validated slices. Compiler-owned rich graph structures are separate from deployed packed types, preventing accidental runtime allocation.

The step API accepts caller-owned output and witness buffers. It returns a small status/result value and never constructs an owned collection.

```
#[repr(transparent)]
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct NodeId(u32);
```



```

#[repr(transparent)]
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub struct ScoreQ(i32);

#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct PackedNode {
    pub child_start: u32,
    pub child_len: u16,
    pub forward_start: u32,
    pub forward_len: u16,
    pub emission_start: u32,
    pub emission_len: u16,
    pub prototype_word_start: u32,
    pub mask_word_start: u32,
    pub radius: u16,
    pub depth: u8,
    pub flags: u8,
}

pub struct GraphView<'a> {
    nodes: &'a [PackedNode],
    edges: &'a [PackedEdge],
    prototypes: &'a [u64],
    masks: &'a [u64],
    routing_ops: &'a [RoutingOp],
    emissions: &'a [EmissionEntry],
}

pub struct RuntimeState<const ACTIVE: usize, const TOP_K: usize> {
    frontier: [ActiveRegion; ACTIVE],
    frontier_len: u8,
    next_frontier: [ActiveRegion; ACTIVE],
    signature: [u64; SIGNATURE_WORDS],
    candidates: TopCandidates<TOP_K>,
}

pub fn step<const ACTIVE: usize, const TOP_K: usize>(
    graph: &GraphView<'_,
    state: &mut RuntimeState<ACTIVE, TOP_K>,
    token: TokenId,
    output: &mut StepOutput<TOP_K>,
    witness: Option<&mut WitnessBuffer>,
) -> Result<(), RuntimeError>;

```

22. Compiler Algorithms

Observation sampling must preserve deterministic resumability. Partition work by content-addressed sample IDs rather than mutable iteration order. Store sufficient metadata to resume without changing the final corpus bytes.

Region induction begins with compiler-side representations and teacher distributions. Candidate partitions are evaluated by predictive entropy reduction, neighbor preservation, stability under bootstrap resampling, runtime probe cost, and artifact size.

Overlaps are retained only when they explain behavior not captured by parent residuals. Explicit intersection nodes should be sparse and justified by held-out gain. Otherwise, multiple membership is represented without an interaction table.

Boolean synthesis searches bounded sparse XOR-polynomial and decision-test candidates. Early tests prioritize broad, stable, high-support separation; later tests refine only within active regions. The compiler emits a deterministic decision DAG and a fallback exact verifier.

Graph packing should use profile-guided layout from compiler statistics: frequently co-active nodes, edges, prototypes, and emissions should be colocated. Packing is deterministic for identical source, corpus, configuration, and compiler version.

23. Next-Token Scoring Model

Scores are quantized log-domain residuals. Let $B(v)$ be the root token prior. Let A be the active semantic cloud, F the predicted next cloud, and X an optional exact residual key. The runtime score is:

$$S(v) = B(v) + \sum[n \text{ in } A] \Delta E(n,v) + \sum[m \text{ in } F] \Delta T(m,v) + \Delta X(X,v).$$

Each residual table is sparse. Candidate tokens are obtained from the union of bounded emission lists associated with active and predicted regions. The runtime does not allocate or score the full vocabulary unless an explicitly certified fallback mode is enabled.

Tie-breaking is canonical: highest score, then lowest token ID or another manifest-declared total ordering. Sampling, when supported, must use a caller-supplied deterministic RNG state and a separately specified integer sampling routine. Greedy prediction is the first required implementation.

24. Proof-Carrying Witness

A prediction witness records the semantic-space CID, graph root CID, input/context code CID or compact code, active region IDs and margins, decision nodes traversed, transition edges applied, score-contributing emission entries, exact residual entry if any, selected token, and operation census.

Witnesses should be bounded and optionally configurable. A compact witness may include Merkle inclusion paths only for externally verifiable or distributed operation; a local witness may reference already validated section roots.

The verifier independently checks graph integrity, routing decisions, region membership distances, edge ranges, score accumulation, tie-breaking, and final result. It does not require the teacher model.

25. Formalized Proof Model

This section gives formal definitions and proof obligations for the deployed runtime. These are design-level proofs under explicit assumptions. They do not prove empirical equivalence to the transformer; that claim belongs to the certification theorem in Section 17.

Assumption A1: GraphView is created only by a validator that proves every slice, offset, range, discriminant, and declared capacity is within the immutable artifact bytes.

Assumption A2: RuntimeState and all optional output buffers are fully allocated before step begins and have capacities matching the artifact limits.

Assumption A3: The implementation of step and all reachable runtime functions contains no allocator invocation, no container growth, and no indirect callback capable of allocating.

Assumption A4: Integer operations use declared wrapping, checked, or saturating semantics, and serialization widths are fixed by the artifact version.

Assumption A5: The safe scalar kernel is the normative semantics. Any architecture-specialized kernel is admitted only after exhaustive or property-based equivalence testing over its finite primitive input domains and differential tests over runtime fixtures.

Theorem 1 - Allocation Freedom

Claim. For every validated graph G , preallocated state S , input token t , and valid caller-owned outputs, execution of $\text{step}(G, S, t)$ performs zero heap allocations.

Proof. By A2, all mutable storage needed by the step already exists. By A3, the transitive call graph contains no allocator entry point and no operation that can grow a collection. GraphView only borrows immutable slices by A1. Fixed arrays, stack locals, and in-place writes do not allocate. Therefore the allocation count during step is zero. This theorem should be enforced by an allocation-counting test harness and by API review of the runtime crate.

Theorem 2 - Operation-Set Conformance

Claim. Every arithmetic or logical operation executed by the normative prediction kernel belongs to $O = \{\text{XOR, AND, OR, shift, rotate, popcount, integer add/subtract, compare, table read}\}$.

Proof. Define the kernel as the only module that performs route distance, score accumulation, and rolling-state updates. Its public primitive methods correspond one-to-one with O , and no multiply, divide, floating-point, or matrix operation is exposed. All prediction functions are implemented solely in terms of these primitives and bounded control flow. Structural induction over the runtime call graph then establishes conformance. A source-level census and disassembly audit are required release checks.

Theorem 3 - Determinism

Claim. Given identical validated artifact bytes, initial runtime state, input token sequence, runtime feature selection, and deterministic tie-breaking/RNG state, the runtime emits identical predictions and witnesses.

Proof. All graph bytes are immutable and all reads are indexed deterministically. Integer primitive semantics are fixed by A4. Frontier truncation and top-K insertion use declared total orderings. There is no data race, clock read, random source, hash-map iteration, or unspecified floating-point reduction. By induction on token steps, equal states and equal inputs produce equal next states and outputs.

Theorem 4 - Bounded Work

Claim. For fixed artifact limits A (active frontier), C (candidate regions per active node), W (signature words per region), E (emission entries per active region), and K (token shortlist), one token step has an explicit finite upper bound independent of total graph size.

Proof. Decision execution is bounded by a manifest-declared maximum D . Region verification examines at most AC candidates and W machine words per candidate. Transition accumulation examines bounded edge ranges, and emission scoring examines at most AE entries with fixed-capacity top- K maintenance. Therefore work is $O(D + ACW + AEK)$ for a simple insertion top- K implementation, or the tighter bound of the selected fixed selection structure. Since A , C , W , E , K , and D are validated constants, runtime work is bounded.

Theorem 5 - Memory Safety of the Safe Runtime

Claim. Under $A1$ and safe Rust compilation, runtime graph reads and state writes cannot access memory outside validated artifact slices or caller-owned buffers.

Proof. `GraphView` fields are immutable slices whose bounds were established during validation. Production runtime code uses safe indexing/iteration or checked access and does not expose mutable aliases to graph bytes. `RuntimeState` has exclusive mutable access for the duration of step. Rust's borrow checker prevents conflicting aliases, and safe slice operations enforce bounds. Therefore out-of-bounds access and data races cannot arise in the safe implementation.

Theorem 6 - Witness Replay Soundness

Claim. If the independent verifier accepts a witness w for graph G and input state S , then the selected token and recorded score equal the result of the normative runtime semantics for the recorded route and evidence entries.

Proof. The verifier recomputes each decision predicate, masked Hamming distance, acceptance radius, edge transition, residual score contribution, and total-order tie-break from validated graph bytes. Acceptance requires equality at every step and a valid inclusion proof or section reference for every evidence item. Therefore an accepted witness corresponds to a valid execution trace of the normative semantics. This does not prove that the graph approximates the teacher well; it proves that the reported result follows from the graph.

Theorem 7 - Forward/Reverse Index Consistency

Claim. For every serialized forward transition edge $e = (u, v, q)$, the reverse index of v contains a reference to the same canonical edge e .

Proof. During packing, forward edges are canonicalized and assigned stable edge IDs. The reverse index is constructed by sorting those same IDs by target node; it does not create independent transition records. Validation checks that every reverse reference resolves to an edge whose target equals the containing reverse range's node. Thus forward and reverse traversal are consistent. Reverse lookup represents predecessor evidence, not mathematical invertibility.

Theorem 8 - Artifact Validation Soundness

If `GraphView` is constructible only after structural validation proves every section, offset, edge target, bytecode instruction, integer width, graph bound, and range is valid, then every normative runtime table access is in bounds under the declared artifact dimensions.

Proof sketch: the validator establishes checked range predicates for every packed reference and produces typed borrowed views rather than unchecked offsets. Runtime iteration is restricted to these validated ranges. Architecture-specific kernels consume the same typed slices and are differential-tested against scalar semantics.

Theorem 9 - Frontier and Candidate Boundedness

For manifest constants F , P , E , and K , every runtime step activates at most F regions, evaluates at most P routing probes, reads at most E emission entries, and retains at most K token candidates.

Proof sketch: all collections are fixed-capacity containers; each insertion has deterministic replacement or rejection semantics; loops are bounded by validated manifest ranges and counters; fallback widening is itself bounded by manifest constants.

Theorem 10 - Residual Non-Duplication

If each region score table contains only its compiler-defined residual relative to its declared baseline and every interaction residual is attached to one canonical overlap node, then replaying a witness adds each committed evidence contribution at most once.

Proof sketch: canonical contribution IDs are unique within a witness, parent residual ancestry is acyclic, and overlap interactions reference a unique node CID. The verifier rejects duplicate contribution IDs or invalid ancestry.

Theorem 11 - Epoch Traceability

For an accepted patch chain whose parent CIDs, tombstones, additions, and route translations validate, every active runtime region and score contribution can be traced to exactly one base or patch epoch and its committed evidence root.

Proof sketch: patch layers are immutable and ordered by parent CID; lookups use deterministic newest-valid precedence; every contribution stores its layer and evidence reference; bounded chain validation rejects forks unless explicitly represented by a newly compacted canonical epoch.

Theorem 12 - Resolution Status Determinism

For a pinned graph, runtime state, and input token, Supported, Boundary, BackedOff, Novel, or Contradictory status is deterministic.

Proof sketch: all status features are derived from deterministic integer distances, margins, support counts, score disagreement, and manifest thresholds with canonical tie-breaking.

26. Empirical Fidelity Theorem

Teacher fidelity cannot be derived solely from the graph format or runtime implementation. It is established by a certificate over pinned evaluation sets.

Define evaluation set $Q = \{(c_i, T(c_i))\}$. Let $R_G(c_i)$ be the graph runtime distribution or scored ranking. Define declared metrics such as top-1 agreement, top-k recall, cross-entropy residual, KL or Jensen-Shannon divergence, repetition rate, semantic-neighbor recall, and route stability.

Conditional Certification Theorem: if the certifier deterministically evaluates the pinned graph and teacher on Q and the resulting metrics satisfy manifest-declared thresholds, then the graph is certified to approximate the teacher to those thresholds on Q and only on the stated evaluation distribution and protocol.

Proof: the certificate contains the CIDs of the teacher, graph, tokenizer, evaluation set, certifier, metric definitions, and raw or committed results. A verifier can reproduce or validate the measurements. Threshold satisfaction follows from those measurements. No claim is made outside Q or beyond the specified statistical confidence and distributional assumptions.

27. Verification and Testing Strategy

Unit tests: packed record parsing, checked range validation, score arithmetic, tie-breaking, rolling-state updates, masked Hamming, top-K insertion, and witness serialization.

Property tests: serializer/parser round trips, canonical bytes, equivalent scalar and optimized kernels, route membership invariants, no duplicate residual contribution, and deterministic packing.

Fuzzing: artifact parser, section validator, witness parser, malformed offsets, integer boundary cases, and unsupported-version handling.

Differential tests: compare graph compiler outputs against a slower reference implementation; compare runtime prediction and witnesses against an executable specification; compare architecture-specific kernels against scalar semantics.

Allocation tests: install a counting allocator in test binaries and assert zero allocation after runtime initialization. Test every prediction API and optional witness mode.

Formal tools: start with Kani for bounded fixed-capacity containers and arithmetic invariants; evaluate Verus or Creusot for runtime proofs; use Lean or Coq only for stable mathematical specifications where the maintenance burden is justified.

Performance tests: benchmark cold and warm graph access, one-token step, sequence generation, routing-only, emission-only, witness generation, and artifact load/validation. Record CPU model, compiler version, target features, cache counters, bytes read, branch misses, and memory bandwidth.

Semantic anti-degeneracy tests: paraphrase consistency, lexical substitutions, polysemy separation, cross-document reuse, tokenizer-boundary variation, rare-context retention, and rate-distortion curves by graph depth.

Novelty and adversarial tests: out-of-distribution inputs, crafted semantic collisions, frontier and candidate exhaustion, fallback-rate stress, patch-layer limits, integer saturation, and unsupported semantic-route use as an identity.

Statistical certification: confidence intervals, bootstrap or concentration method, per-slice and worst-group metrics, repeated-seed stability, and explicit stopping rules.

28. Milestone Plan

Milestone 0 - Baseline and contracts (1-2 weeks): freeze terminology; define teacher, graph, runtime, certificate, novelty, and fallback boundaries; benchmark current allocations, operations, cache behavior, and bytes read.

Milestone 1 - Packed graph and trusted parser (2-4 weeks): implement R4G1 sections; two-stage validation; memory-mapped GraphView; fuzzing; canonical round trips; checked offset and bytecode invariants.

Milestone 2 - Overlapping region prototype (3-5 weeks): reuse current context signatures; calibrate radii; activate bounded multiple classes; build parent and overlap graph; retain nearest-class fallback; measure frontier widths and routing recall.

Milestone 3 - Semantic anti-degeneracy (3-6 weeks): add multi-view perturbations, paraphrase and polysemy suites, minimum-description-length objective, predictive-sufficiency curves, and semantic-coherence certificate.

Milestone 4 - Semantic transition and residual emission (3-5 weeks): compile forward and evidential-predecessor indexes, root priors, child residuals, interaction residuals, shared emission blocks, and bounded token shortlists.

Milestone 5 - Allocation-free graph runtime (2-4 weeks): fixed-capacity RuntimeState, multi-timescale state skeleton, caller-owned outputs, scalar native kernel, operation census, zero-allocation tests, novelty status, and witness replay.

Milestone 6 - Boolean routing synthesis (4-8 weeks): decision DAGs and sparse XOR-polynomial candidate search; exact verifier; measured top-M recall; deterministic widening and bounded exhaustive fallback.

Milestone 7 - Hardware-aware packing (3-6 weeks): profile-guided node ordering, hot/cold splitting, cache-line alignment, shared blocks, byte-read optimization, and architecture-neutral layout certificate.

Milestone 8 - Long-context state (4-8 weeks): compile local, segment, and session state updates; entity reactivation and dependency-distance suites; fixed-capacity saturation behavior.

Milestone 9 - Epochs and patches (3-6 weeks): immutable deltas, tombstones, provenance deletion, bounded layer lookup, compaction, and route-translation evidence.

Milestone 10 - Formal verification tranche (4-8 weeks): executable specification, Kani or equivalent proofs for containers/ranges, artifact validator properties, residual uniqueness, reverse-index consistency, deterministic replay, and proof-status matrix.

Milestone 11 - Architecture acceleration (ongoing): AVX2, AVX-512, and NEON adapters behind runtime dispatch; differential equivalence; cache and NUMA tuning; performance certificate profiles.

29. Acceptance Gates

Gate A - Correctness: graph output matches the reference runtime exactly for all regression fixtures; malformed artifacts are rejected; witnesses replay.

Gate B - Runtime contract: zero allocations per token; no floating point or multiply in normative kernel; no locks; bounded frontier and probes; scalar implementation passes Miri-compatible tests where applicable.

Gate C - Fidelity: overlapping graph is no worse than the current compiled baseline on declared held-out metrics before replacing it as the default.

Gate D - Performance: synthesized graph reduces instructions, bytes read, or latency per token without violating fidelity thresholds. Performance claims require statistically stable benchmark evidence.

Gate E - Reproducibility: identical pinned inputs and compiler configuration produce byte-identical graph and certificate artifacts.

Gate F - Proof status: every theorem and assumption has an implementation link, automated test, formal-tool result, or explicitly documented unproven status. No theorem may be described as machine-verified unless the proof artifact is complete and checked in CI.

Gate G - Semantic validity: regions demonstrate cross-context reuse, perturbation stability, predictive sufficiency, and anti-memorization behavior before being described as semantic.

Gate H - Routing safety: candidate shortlisting meets declared recall and fallback limits; no optimized router replaces the normative verifier without equivalent evidence.

Gate I - Novelty and safety: unsupported inputs are detected and handled according to the manifest; adversarial inputs cannot exceed fixed frontier, probe, candidate, or patch-layer bounds.

Gate J - Lifecycle: graph patches are immutable, traceable, deletable by provenance, and translatable across epochs.

Gate K - Statistical rigor: every empirical quality claim includes distribution, sample size, confidence bounds, slices, and reproducible protocol.

30. Risks and Mitigations

Semantic collapse: clusters may preserve representation geometry but lose prediction behavior.

Mitigation: optimize jointly for representation, teacher distribution, and transition coherence; certify held-out predictive residuals.

Graph explosion: overlapping covers and intersections may grow combinatorially. Mitigation: fixed support/gain thresholds, bounded memberships, sparse materialization, residual pruning, and artifact-byte penalties in the compiler objective.

Double counting: several active regions may repeat the same teacher evidence. Mitigation: root-plus-residual decomposition and explicit interaction residuals only when supported.

Poor long-range behavior: a short rolling state may omit distant dependencies. Mitigation: compile stateful region transitions, multi-timescale lanes, bounded persistent features, and held-out long-context suites.

Boolean approximation error: XOR-polynomial routing may distort compiler-side regions. Mitigation: shortlist plus exact masked-Hamming verifier, fallback route, and measured routing recall.

SIMD unsafety or nondeterminism: architecture-specific code may diverge. Mitigation: safe scalar norm, isolated adapters, strict equivalence tests, and optional disabling of specialized kernels.

Overclaiming proof or model quality: structural runtime proofs do not imply semantic quality. Mitigation: maintain a public proof-status matrix and separate mathematical invariants from empirical certificates.

Semantic memorization: regions may become compressed phrase tables. Mitigation: multi-view invariance, minimum-description-length penalties, cross-context reuse thresholds, and semantic-coherence certification.

Novelty overconfidence: every input has a nearest prototype even when unsupported. Mitigation: calibrated radii, support and disagreement signals, explicit novelty status, and deterministic fallback or abstention.

Candidate false negatives: a fast router may prune correct regions. Mitigation: normative exact verifier, top-M recall certification, confidence-triggered widening, and bounded exhaustive fallback.

Memory-bandwidth bottleneck: native arithmetic may be hidden by scattered reads. Mitigation: hardware-aware packing, byte-read objectives, profile-guided layout, and cache-counter acceptance gates.

Tokenizer artifacts: source BPE boundaries may be mistaken for concepts. Mitigation: contextual spans, byte anchors, tokenizer-variation tests, and model-specific lexicalization adapters.

Patch-chain complexity: continual updates may degrade boundedness and reproducibility. Mitigation: immutable bounded layers, canonical compaction, route translation, and chain validation.

Bias or unsafe amplification: graph compression may strengthen source-model failures. Mitigation: amplification and erasure metrics, per-slice certification, membership-edge provenance, and deletion support.

Adversarial semantic collisions: crafted inputs may target hot regions or exhaust work budgets. Mitigation: strict bounds, parser and bytecode fuzzing, checked arithmetic, adversarial suites, and no authorization use of semantic routes.

31. Immediate Issue Backlog

Define R4G1 wire-format RFC and GraphView validation invariants.

Introduce fixed-width newtypes and eliminate semantic use of `usize` in serialized compiler/runtime structures.

Add counting-allocator tests around the existing transformerless Runtime APIs.

Instrument current context assignment with per-token operation and candidate-scan counts.

Add compiler output for per-class Hamming distance distributions and calibrated radii.

Implement bounded multi-membership assignment while preserving nearest-class fallback.

Create a reference ActiveFrontier and packed edge range implementation.

Compile forward region transitions and reverse indexes from the existing teacher corpus.

Replace floating semantic route scores in deployed paths with ScoreQ fixed-point residuals.

Create certificate schema with source, corpus, graph, metric, operation, and benchmark CIDs.

Add deterministic artifact rebuild test to CI.

Write the executable proof model for allocation freedom assumptions, bounded ranges, deterministic top-K, and reverse-index construction.

Add semantic anti-degeneracy corpus transformations and evaluation harness.

Implement predictive-sufficiency and rate-distortion reports by graph depth.

Add explicit ResolutionStatus and manifest-defined fallback policy.

Measure shortlist top-M recall and fallback behavior against the reference classifier.

Add bytes-read, cache-miss, and branch-miss counters to performance certificates.

Design multi-timescale fixed-capacity RuntimeState for local, segment, and session clouds.

Define tokenizer-neutral span and byte anchors in the observation schema.

Draft immutable graph patch and route-translation RFC.

Add source-bias amplification, rare-group erasure, and provenance-deletion tests.

Threat-model semantic collisions, frontier exhaustion, candidate explosion, and integer saturation.

Specify behavioral graph equivalence and confidence-bounded empirical claims.

32. Decision Record

Decision: semantics are discovered from trained model behavior rather than supplied as a supervised ontology.

Decision: contextual occurrences are clustered before lexical identities; tokens may have multiple context-dependent memberships.

Decision: graph regions are multiresolution, overlapping, and proof-addressable; they are not semantic atoms.

Decision: content CIDs and semantic route codes remain separate.

Decision: runtime inference uses packed immutable tables and fixed-capacity state, not object graphs or dynamic containers.

Decision: next-token prediction is decomposed into semantic transition, sparse lexical emission, and exact residual evidence.

Decision: scalar safe Rust defines normative semantics; optimized CPU kernels are replaceable implementations subject to equivalence testing.

Decision: mathematical runtime guarantees and empirical teacher-fidelity claims are documented and certified separately.

Decision: predictive coherence alone is insufficient to call a region semantic; multi-view invariance, reuse, and predictive sufficiency are required.

Decision: optimized Boolean routing only shortlists; masked-Hamming verification remains normative until an equivalence proof exists.

Decision: runtime returns explicit supported, boundary, backed-off, novel, or contradictory status.

Decision: long-range information is represented by bounded multi-timescale states rather than an unbounded token history.

Decision: compiler layout is optimized for bytes read and cache behavior, not arithmetic count alone.

Decision: source token IDs are lexicalization details, not permanent semantic identities.

Decision: updates use immutable content-addressed epochs and patches with bounded lookup and route translation.

Decision: graph-quality certificates include statistical uncertainty, distribution slices, and amplification or erasure analysis.

33. References

[R1] UOR Foundation, uor-r4, feature/proof-carrying-semantic-routing branch and transformerless compiler/runtime sources, accessed July 2026.

[R2] Project Rust best-practices document supplied by the user:
<https://gist.githubusercontent.com/auser/c3161f55a8393faa8af5ddda68c6bafa/raw/f87e449a41f01ebc76ac2249e42564b1b5860b0c/rust-best-practices.md>

[R3] Rust API Guidelines: <https://rust-lang.github.io/api-guidelines/X>

[R4] Rust Style Guide: <https://doc.rust-lang.org/stable/style-guide/X>

[R5] Atomic Language Model repository, used only as a research comparison for compact feature grammar and proof ambitions: <https://github.com/dkypuros/atomic-lang-modelX>

[R6] Noam Chomsky, formal language hierarchy and recursive grammar concepts; used as structural background, not as supervised semantic labeling.

[R7] Prototype theory, frame semantics, construction grammar, formal concept analysis, and graph-directed fractal systems; used as conceptual evaluation frameworks rather than mandatory runtime dependencies.